30-Day PostgreSQL Course

Day 1

What is PostgreSQL?

Session Info	Details
Duration	60 minutes
Level	Complete Beginner — no prior SQL knowledge required
Goal	Understand what PostgreSQL is, how it fits into data pipelines, and get it running on your machine
Week	Week 1 — SQL Foundations (Days 1–7)

What you will be able to do after this session:

✓ Explain what a relational database is in plain language

✓ Describe where PostgreSQL fits in a real data pipeline

✓ Install PostgreSQL on your machine

✓ Connect using the psql command-line client

✓ Run your very first database commands

Session Agenda

Use this as your recording script. Each section maps to a timed segment.

Segment	Time
1. Introduction & What is a Database?	0:00 – 0:10
2. What is PostgreSQL?	0:10 – 0:20
3. PostgreSQL in a Data Pipeline	0:20 – 0:30
4. Installation Walkthrough	0:30 – 0:45
5. Hands-On Exercise	0:45 – 0:55
6. Recap & Day 2 Preview	0:55 – 1:00

■ Segment 1 — Introduction & What is a Database? (0:00 – 0:10)

1. What is a Database?

Opening Hook — say this on camera

Script:

Before we write a single line of SQL, let's answer the most important question: what problem are we actually solving?

Imagine you work at an e-commerce company. Every second, customers are placing orders, clicking products, and creating accounts. That data needs to live somewhere — somewhere reliable, fast, and queryable. That's what a database is for.

Key Concept: What is a Relational Database?

A relational database organizes data into tables — like spreadsheets, but much more powerful.

Think of it this way:

- A spreadsheet is great for 100 rows of data you manage manually
- A database handles millions of rows, updated by multiple systems simultaneously
- Databases let you link tables together — that's what "relational" means

Example — a simple e-commerce setup (two linked tables):

customers table	orders table
`id | name | email`	`id | customer_id | amount`
`1 | Alice | alice@email.com`	`101 | 1 | 59.99`
`2 | Bob | bob@email.com`	`102 | 1 | 23.50`

Notice: orders.customer_id = 1 links back to Alice in the customers table. That's a relationship.

■ Segment 2 — What is PostgreSQL? (0:10 – 0:20)

2. What is PostgreSQL?

Definition

PostgreSQL (pronounced post-gres-Q-L, or just "Postgres") is a free, open-source relational database system. It has been in active development since 1986 — nearly 40 years — making it one of the most battle-tested databases in the world.

Why PostgreSQL for Data Engineering?

Talking point — say this on camera:

There are many databases out there — MySQL, SQLite, Oracle, SQL Server. So why are we learning PostgreSQL specifically?

Because it's the industry standard for data engineering. The most popular cloud data warehouses today — Amazon Redshift, Aurora, Supabase — are all either built on Postgres or heavily inspired by it.

Feature	Why it matters for data engineering
Open source & free	No license costs — used everywhere from startups to Fortune 500
ACID compliant	Your data won't get corrupted during pipeline failures
Advanced SQL	CTEs, window functions, JSON support — all built in
Extensible	Add custom types, functions, and extensions like PostGIS
Large ecosystem	Supported by every major cloud provider and BI tool

■ Segment 3 — PostgreSQL in a Data Pipeline (0:20 – 0:30)

3. PostgreSQL in a Data Pipeline

The Big Picture

As a data engineer, you won't just query existing databases. You'll build pipelines — automated flows that move, clean, and transform data. Here's where PostgreSQL fits:

Pipeline Flow (draw this on a whiteboard or show the diagram):

RAW SOURCES → PostgreSQL → CONSUMERS

Raw sources: App events, CSV files, APIs, other databases

PostgreSQL: Store, clean, join, aggregate, and transform data

Consumers: Dashboards, reports, ML models, downstream pipelines

Three Roles PostgreSQL Plays in Pipelines

Role 1 — Staging Layer

Raw data lands here first, exactly as it arrived — messy column names, duplicate rows, nulls everywhere. You load it fast and fix it later.

Role 2 — Transformation Layer

SQL queries reshape the data: joins, aggregations, deduplication, business logic. This is where most of your SQL skills will be applied.

Role 3 — Serving Layer

Clean, transformed tables that dashboards and applications read from. Often exposed via views or materialized views (covered in Week 4).

Data Engineering connection:

Every major concept in this course — CTEs, window functions, upserts, transactions, indexes — exists because real pipelines need them.

By Day 30, you'll build a working incremental pipeline entirely in PostgreSQL.

■ Segment 4 — Installation Walkthrough (0:30 – 0:45)

4. Installing PostgreSQL

Walk through the steps on camera for your operating system. Show the terminal output after each command.

macOS (Homebrew) — Recommended

```
# Step 1: Install Homebrew if you don't have it

/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"


# Step 2: Install PostgreSQL 16

brew install postgresql@16


# Step 3: Start the PostgreSQL service

brew services start postgresql@16


# Step 4: Add psql to your PATH

echo 'export PATH="/opt/homebrew/opt/postgresql@16/bin:$PATH"' >> ~/.zshrc

source ~/.zshrc


# Step 5: Verify installation

psql --version
```

Windows

- Go to: https://www.postgresql.org/download/windows/
- Download the installer for PostgreSQL 16
- Run the installer — accept all defaults
- Set a password for the 'postgres' superuser (remember this!)
- Open 'SQL Shell (psql)' from the Start menu

Linux (Ubuntu / Debian)

```
sudo apt update

sudo apt install postgresql postgresql-contrib

sudo systemctl start postgresql

sudo systemctl enable postgresql


# Switch to the postgres user to connect

sudo -u postgres psql
```

Connecting with psql

psql is the official PostgreSQL command-line client — your terminal window into the database.

```
# Connect to the default 'postgres' database

psql postgres


# You'll see this prompt — you're in!

postgres=#
```

Essential psql Meta-Commands

These commands start with a backslash. They are psql commands, not SQL — they don't need a semicolon.

Command	What it does
\l	List all databases on the server
\dt	List all tables in the current database
\c dbname	Switch to a different database
\d tablename	Describe a table's columns and types

\q	Quit psql
\?	Show all available psql commands

Common gotcha to mention on camera:

SQL commands need a semicolon at the end.

psql meta-commands (starting with \) do NOT need a semicolon.

This trips up almost every beginner — call it out clearly!

■ Segment 5 — Hands-On Exercise (0:45 – 0:55)

5. Hands-On Exercise

> **Tell your viewers:**
>
> Pause the video here and complete these steps on your own machine.
>
> Come back once you've seen the output. If you hit an error, I'll cover the most common ones next.

Exercise Steps

Step 1 — Open your terminal (or SQL Shell on Windows)

Step 2 — Connect to PostgreSQL:

```
psql postgres
```

Step 3 — Run the list databases command:

```
\l
```

You should see at least 3 databases: postgres, template0, template1

Step 4 — Run the list tables command:

```
\dt
```

Expected: 'Did not find any relations.' — that's fine, no tables yet!

Step 5 — Check your PostgreSQL version (your first SQL command!):

```
SELECT version();
```

Don't forget the semicolon — this is SQL, not a psql meta-command.

Step 6 — Exit psql:

```
\q
```

Expected Output

```
$ psql postgres
```

```
psql (16.x)
Type "help" for help.

postgres=# \l
List of databases
Name | Owner | Encoding | ...
-----------+-------+----------+---
postgres | ... | UTF8 | ...
template0 | ... | UTF8 | ...
template1 | ... | UTF8 | ...

postgres=# \dt
Did not find any relations.

postgres=# SELECT version();
version
-----------------------------------------
PostgreSQL 16.x on ...
(1 row)
```

Common Errors & Fixes

Error Message	Fix
command not found: psql	Re-run the PATH export command from the install steps
could not connect to server	Service not running — run: brew services start postgresql@16

role 'your_username' does not exist	Run: createuser -s $(whoami) in your terminal
password authentication failed	On Windows, use the password you set during installation

■ Segment 6 — Recap & Day 2 Preview (0:55 – 1:00)

6. Recap & Day 2 Preview

What We Covered Today

- A relational database stores data in linked tables — rows and columns
- PostgreSQL is the industry-standard open-source database for data engineering
- In a pipeline, Postgres acts as the staging, transformation, and serving layer
- psql is the command-line client — your window into the database
- \l lists databases, \dt lists tables, SELECT version(); is your first SQL

Day 2 Preview — Your First SELECT

End with this on camera:

Tomorrow, we write real SQL.

You'll learn the SELECT and FROM keywords, how to choose which columns to return, and how to alias columns with AS — one of the most used patterns in data engineering.

Make sure your PostgreSQL is installed and working before then. See you in Day 2.

Appendix — Quick Reference Card

Keep this open during recording for quick reference.

psql command	Description
psql postgres	Connect to the postgres database
psql -U username dbname	Connect with a specific username
\l	List all databases
\dt	List tables in current database
\d tablename	Describe a table's structure
\c dbname	Switch database
\q	Quit psql
SELECT version();	Check PostgreSQL version (SQL — needs semicolon)